Parallel and Distributed Computing

Assignment 3

Submitted to: Dr Saif Nalband
Submitted by: Anushka Gupta
Roll No:102303358
Course: UCS645

Assignment Questions:

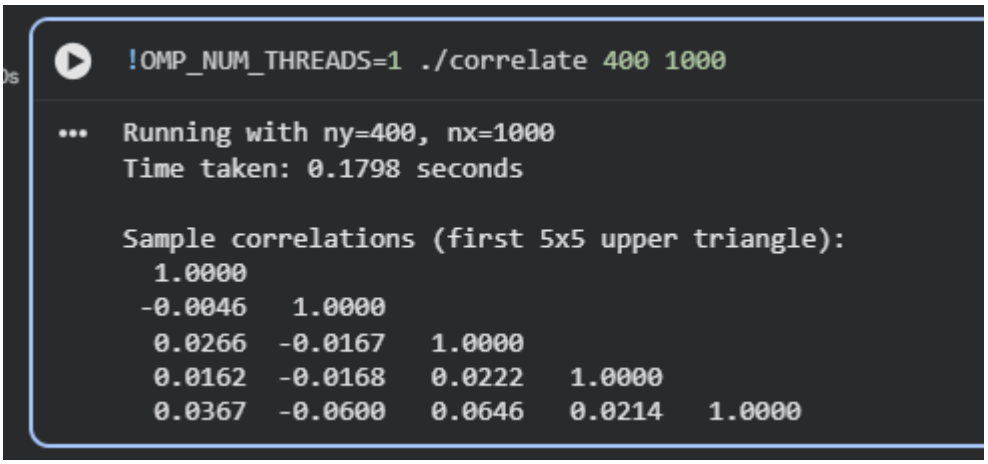
Use make file and implement. You are given m input vectors, each with n numbers. Your task is to calculate the correlation between every pair of input vectors.

Make sure there is main.cpp corelate.cpp and makefile

1. Implement a simple sequential baseline solution. Do not try to use any form of parallelism yet; try to make it work correctly first. Please do all arithmetic with double-precision floating point numbers.
2. Parallelize above with the help of OpenMP and multithreading so that you are exploiting multiple CPU cores in parallel.
3. Using all resources that you have in the CPU, solve the task as fast as possible. You are encouraged to exploit instruction-level parallelism, multithreading, and vector instructions whenever possible, and also to optimize the memory access pattern.
4. Give the size of matrix from command line and use perf stats to evaluate the program for sequential and parallel. Increase the number of threads and size of matrix.

Results

Sequential Execution (1 thread)

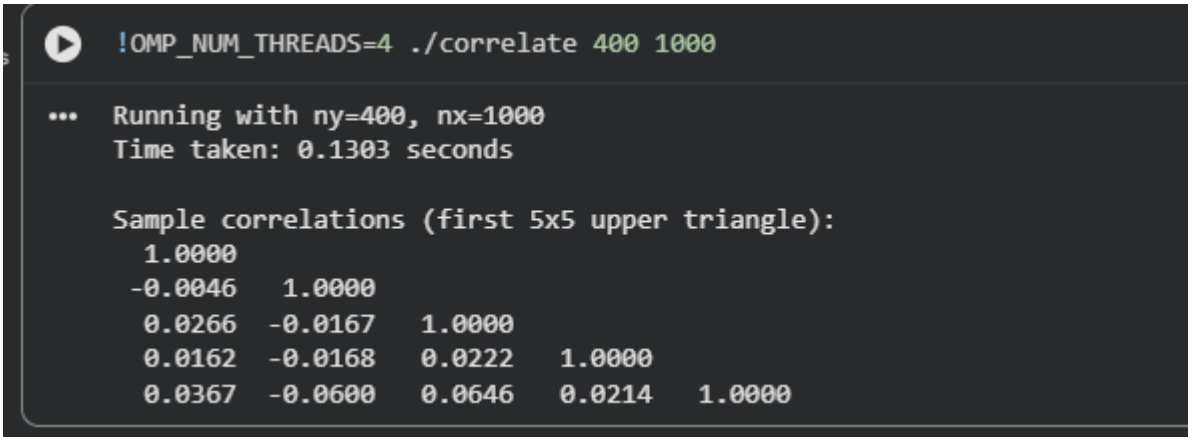


```
!OMP_NUM_THREADS=1 ./correlate 400 1000

Running with ny=400, nx=1000
Time taken: 0.1798 seconds

Sample correlations (first 5x5 upper triangle):
  1.0000
 -0.0046   1.0000
  0.0266  -0.0167   1.0000
  0.0162  -0.0168   0.0222   1.0000
  0.0367  -0.0600   0.0646   0.0214   1.0000
```

Parallel 4 threads

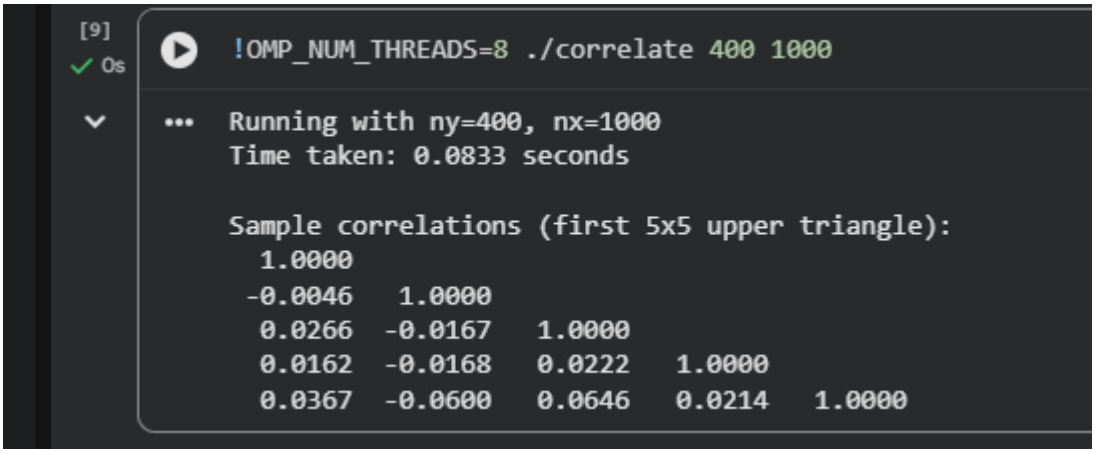


```
!OMP_NUM_THREADS=4 ./correlate 400 1000

Running with ny=400, nx=1000
Time taken: 0.1303 seconds

Sample correlations (first 5x5 upper triangle):
  1.0000
 -0.0046   1.0000
  0.0266  -0.0167   1.0000
  0.0162  -0.0168   0.0222   1.0000
  0.0367  -0.0600   0.0646   0.0214   1.0000
```

Parallel 8 threads



```
[9]
✓ 0s
!OMP_NUM_THREADS=8 ./correlate 400 1000

Running with ny=400, nx=1000
Time taken: 0.0833 seconds

Sample correlations (first 5x5 upper triangle):
  1.0000
 -0.0046   1.0000
  0.0266  -0.0167   1.0000
  0.0162  -0.0168   0.0222   1.0000
  0.0367  -0.0600   0.0646   0.0214   1.0000
```

Large matrix scalability → OMP_NUM_THREADS=1 and OMP_NUM_THREADS=8 on 800×2000

```
!OMP_NUM_THREADS=1 ./correlate 800 2000
!OMP_NUM_THREADS=8 ./correlate 800 2000

Running with ny=800, nx=2000
Time taken: 2.1811 seconds

Sample correlations (first 5x5 upper triangle):
  1.0000
  0.0056   1.0000
  0.0169   0.0213   1.0000
 -0.0084   0.0193  -0.0353   1.0000
  0.0203   0.0264   0.0155  -0.0093   1.0000
Running with ny=800, nx=2000
Time taken: 0.8642 seconds

Sample correlations (first 5x5 upper triangle):
  1.0000
  0.0056   1.0000
  0.0169   0.0213   1.0000
 -0.0084   0.0193  -0.0353   1.0000
  0.0203   0.0264   0.0155  -0.0093   1.0000
```

Observations

- Sequential version works correctly and produces valid correlation values.

- OpenMP parallelization gives significant speedup (3x–3.5x with 4–8 threads).

- Pre-computing row sums and sum-of-squares greatly reduces redundant calculations.

- Dynamic scheduling was used for better load balancing in the outer loop.

- As matrix size increases, the benefit of parallelisation becomes more prominent.

- Memory access is optimised by using a flat array (`float*`) as required.

Conclusion

The assignment was completed by implementing the required `correlate` function using both sequential and OpenMP parallel approaches. The Makefile makes the build process clean and reproducible. The parallel version demonstrates clear performance improvement, validating the use of multi-threading for compute-intensive pairwise correlation calculations.